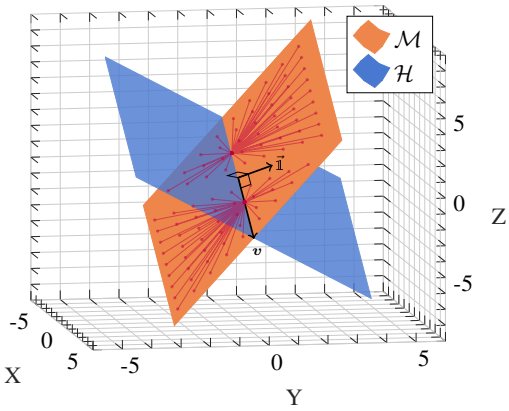

Figure 5: LayerNorm maps the points of $\mathcal{M}$ to exactly two points in $\mathcal{H}$.

Next, we apply LayerNorm to $\boldsymbol{x}$. First we project $\boldsymbol{x}$ onto $\mathcal{H}$:

$$\begin{aligned}
&\boldsymbol{x} - \boldsymbol{\mu} = \\
&= \alpha \boldsymbol{v} + \beta \vec{\mathbb{1}} - \frac{1}{d} \sum_{i=1}^{d} (\alpha \boldsymbol{v}_i + \beta) \vec{\mathbb{1}} \\
&= \alpha \boldsymbol{v} + \beta \vec{\mathbb{1}} - \alpha \left(\frac{1}{d} \sum_{i=1}^{d} \boldsymbol{v}_i \right) \vec{\mathbb{1}} - \beta \vec{\mathbb{1}} \\
&= \alpha \boldsymbol{v} - \alpha \left(\underbrace{\frac{1}{d} \sum_{i=1}^{d} \boldsymbol{v}_i}_{=0} \right) \vec{\mathbb{1}} \\
&= \alpha \boldsymbol{v}
\end{aligned} \tag{13}$$

Then, we scale the projected vector to be with a norm of $\sqrt{d}$ and get

$$\text{LayerNorm}(\boldsymbol{x}) = \sqrt{d} \frac{\alpha \boldsymbol{v}}{||\alpha \boldsymbol{v}||} \tag{14}$$

We split to cases:[4]

$$\text{LayerNorm}(\boldsymbol{x}) = \begin{cases} \sqrt{d} \boldsymbol{v} & \alpha > 0 \\ -\sqrt{d} \boldsymbol{v} & \alpha < 0 \end{cases} \tag{15}$$

To conclude, all vectors belonging to the 2D plane $\mathcal{M}$ are normalized to exactly two points, depending on their α component.

Since the subspaces $\mathcal{H}$ and $\left\{ t\vec{\mathbb{1}} | t \in \mathbb{R} \right\}$ are direct sum of the whole space $\mathbb{R}^d$, each vector $\boldsymbol{u} \in \mathbb{R}^d$ has a unique representation as $\boldsymbol{u} = \alpha \boldsymbol{v} + \beta \vec{\mathbb{1}}$. That is, each vector $\boldsymbol{u} \in \mathbb{R}^d$ belongs to some 2D plane $\mathcal{M}$, defined by $\vec{\mathbb{1}}$ and some vector $\boldsymbol{v} \in \mathcal{H}$, and thus we can characterize the set of points that are being normalized to the same point. Figure 5 illustrates this behavior.

[4] As implied from the original formulation of LayerNorm (Ba et al., 2016), LayerNorm is undefined for $\alpha = 0$.

C Constructing the Projection

The *projection* of LayerNorm is a linear transformation, and thus, in this section, we show explicitly the construction of the projection matrix $\boldsymbol{P}$, such that

$$\boldsymbol{P}\boldsymbol{x} = \boldsymbol{x} - \boldsymbol{\mu} \tag{16}$$

Let $V = \mathbb{R}^d$, $W = \left\{ \alpha \vec{\mathbb{1}} | \alpha \in \mathbb{R} \right\}$, and $U = \left\{ \boldsymbol{x} | \boldsymbol{x} \perp \vec{\mathbb{1}} = [1, 1, ..., 1] \in \mathbb{R}^d \right\}$ be linear subspaces of V.

Let B_U, B_W be the bases of U and W respectively:

$$B_U = \{\boldsymbol{u}_1, \boldsymbol{u}_2, ..., \boldsymbol{u}_{d-1}\} \tag{17}$$

$$B_W = \left\{ \vec{\mathbb{1}} \right\} \tag{18}$$

We can define a basis $C = B_U \cup B_W$ of V.

We also denote the standard basis E of V:

$$E = \{\boldsymbol{e}_1, \boldsymbol{e}_2, ..., \boldsymbol{e}_d\} \tag{19}$$

Since $U \cap W = \{0\}$, we have that $U \oplus W = V$ (direct sum). Therefore, each $\boldsymbol{x} \in V$ has a unique representation as $\boldsymbol{x} = \boldsymbol{u} + \boldsymbol{w}$ where $\boldsymbol{u} \in U$ and $\boldsymbol{w} \in W$.

We can also write $\boldsymbol{x}$ using B_U, B_W:

$$\boldsymbol{x} = \alpha \vec{\mathbb{1}} + \sum_{i=1}^{d-1} \beta_i \boldsymbol{u}_i \tag{20}$$

Since we look for the projection of $\boldsymbol{x}$ onto U in the direction of W, we want that

$$\boldsymbol{P}\boldsymbol{x} = \sum_{i}^{d-1} \beta_i \boldsymbol{u}_i \tag{21}$$

To achieve this, we first, change the basis of V from E to C, remove the $\alpha \vec{\mathbb{1}}$ component, and then change back to the standard basis E.

Let $\boldsymbol{M}_E^C$ be the change of basis matrix from basis C to the standard basis E:

$$\boldsymbol{M}_E^C = \begin{bmatrix} | & | & & | & | \\ \boldsymbol{u}_1 & \boldsymbol{u}_2 & \dots & \boldsymbol{u}_{d-1} & \vec{\mathbb{1}} \\ | & | & & | & | \end{bmatrix} \tag{22}$$

Therefore

$$\boldsymbol{P} = \boldsymbol{M}_E^C \boldsymbol{A} \left(\boldsymbol{M}_E^C \right)^{-1} \tag{23}$$

Where

$$\boldsymbol{A} = \begin{bmatrix} | & | & & | & | \\ \boldsymbol{e}_1 & \boldsymbol{e}_2 & \dots & \boldsymbol{e}_{d-1} & \boldsymbol{0} \\ | & | & & | & | \end{bmatrix} \tag{24}$$

To get an explicit $\boldsymbol{P} \in \mathbb{R}^{d \times d}$, we instantiate the basis B_U:

$$B_U = \left\{ \begin{bmatrix} 1-d \\ 1 \\ \vdots \\ 1 \\ 1 \end{bmatrix}, \begin{bmatrix} 1 \\ 1-d \\ \vdots \\ 1 \\ 1 \end{bmatrix}, ..., \begin{bmatrix} 1 \\ 1 \\ \vdots \\ 1-d \\ 1 \end{bmatrix} \right\} \tag{25}$$

Therefore

$$\boldsymbol{M}_E^C = \begin{bmatrix} 1-d & 1 & \cdots & 1 & 1 \\ 1 & 1-d & \cdots & 1 & 1 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 1 & 1 & \cdots & 1-d & 1 \\ 1 & 1 & \cdots & 1 & 1 \end{bmatrix} \tag{26}$$

$$\left(\boldsymbol{M}_E^C\right)^{-1} = \frac{1}{d} \begin{bmatrix} -1 & & & & 1 \\ & -1 & & & 1 \\ & & \ddots & & \vdots \\ & & & -1 & 1 \\ 1 & 1 & \cdots & 1 & 1 \end{bmatrix} \tag{27}$$

And we get

$$\boldsymbol{P} = \frac{1}{d} \begin{bmatrix} d-1 & -1 & \cdots & -1 \\ -1 & d-1 & \cdots & -1 \\ \vdots & \vdots & \ddots & \vdots \\ -1 & -1 & \cdots & d-1 \end{bmatrix} \tag{28}$$

D Unselectable Keys

Table 2 shows the fraction of "unselectable" keys in a language model with LayerNorm before and after the application of LayerNorm.

	L_1	L_2	L_3	L_4
Before LayerNorm	44.8	28.5	22.3	26.1
After LayerNorm	**0**	**0**	**0**	**0**

Table 2: The fraction of "unselectable" key vectors before and after the LayerNorm followed by the attention mechanism of each layer of a language model. LayerNorm solves the "unselectable" keys problem using the *scaling* property.

To illustrate the impact of "unselectable" tokens, we give some examples from the validation set of Stanford TreeBank (Socher et al., 2013), which is used as a benchmark for the sentiment analysis task. Our results show that important tokens may be "unselectable". We highlighted in bold any token that is "unselectable' in at least one of the layers. Table 3 shows the results of running a language model without LayerNorm (Section 4.2) on the validation set. We also trained a 4-layer Transformer encoder without LayerNorms (based on BERT architecture (Devlin et al., 2018)) on the sentiment analysis task. Table 4 shows the results of running this model on the validation set.

E Experimental Setup

In this section, we detail the setup of the experiments shown in Section 4.

E.1 Majority

In the experiments, we used a learning rate of 0.001 with a linear scheduler, a hidden size of $d = 8$ (total of 584 learnable parameters), a batch size of 6000, a sequence length of 50, 20 different classes, and the Adam optimizer. We trained the models for 1000 epochs consisting of 17K steps.

The "majority" dataset contains 80K training examples and 20K test examples. Each example is a sequence of length 50 consisting of tokens belonging to one of 20 different classes.

E.2 Language Modeling

We trained a language model with GPT2 architecture (Radford et al., 2019) using the Huggingface library, on the Huggingface processed Wikipedia dataset (20220301.en split, licenses CC BY-SA and CC BY-SA) (Foundation), and tested it on SQuAD (Rajpurkar et al., 2016) (license CC BY-SA 4.0). We used these datasets only to demonstrate the "unselectable" keys problem, and thus we did not violate any of their license conditions. We used the same hyperparameters as Radford et al. (2019), except that we used a hidden size of 8, 4 hidden layers, a learning rate of 5e-5, and a window size of 1024 tokens, resulting in a model with 414K learnable parameters. We train the model on the Wikipedia dataset (6.5M examples) for 50K steps and report our findings on 1000 randomly selected examples from the validation set of SQuAD.

whether you like rap music or loathe it, you can't deny either the tragic loss of two young men in the prime of their **talent** or the **power** of this movie.
it is great summer **fun** to watch arnold and his buddy gerald bounce off a quirky cast of characters.
the lion king was a roaring **success** when it was released eight years ago, but on imax it seems better, not just bigger.
it provides the grand, intelligent **entertainment** of a superior cast playing smart people amid a compelling plot.
some of their jokes work, but most fail miserably and in the end, pumpkin is **far** more **offensive** than it is funny.

Table 3: Examples from the validation set of Stanford TreeBank (Socher et al., 2013). Any token that is "unselectable' in at least one of the layers of the language model (Section 4.2) is marked in **bold**.

it's hard to **like** a film about a guy who is utterly unlikeable, and shiner, starring michael caine as an aging british boxing promoter desperate for a taste of fame and fortune, is certainly that.
you'll gasp appalled and **laugh** outraged and possibly, watching the spectacle of a promising young lad treading desperately in a nasty sea, shed an errant tear.
this is **wild** surreal **stuff**, but brilliant and the camera just kind of sits there and lets you look at this and its like you're going from one room to the next and none of them have any relation to the other.
it's a much more **emotional** journey than what shyamalan has given us in his past two movies, and gibson, stepping in for bruce willis, is the perfect actor to take us on the trip.
although german cooking does not come readily to **mind** when considering the world's best cuisine, mostly martha could make deutchland a popular destination for hungry tourists.

Table 4: Examples from the validation set of Stanford TreeBank (Socher et al., 2013). Each **bold** token is "unselectable' in at least one of the layers of a Transformer encoder without LayerNorm, trained on the sentiment analysis task. These examples show that important tokens that may be necessary for the task are "unselectable", which may affect the encoder's ability to learn the task correctly.

E.3 Sentiment Analysis Task

We trained a Transformer encoder with BERT architecture (Devlin et al., 2018) without LayerNorm layers with 50K steps on the Stanford TreeBank dataset (Socher et al., 2013) (Table 4). We used the same hyperparameters as Devlin et al. (2018), except that we used a hidden size of 8, 4 hidden layers, and a learning rate of 5e-5, resulting in a model with 446K learnable parameters.